区块链创新应用方向

设计报告

ChainGuard

面向 AI Agent 协作的
区块链授权与审计系统

跨组织共同治理　逐动作可信授权　分级可验证审计

2026 年 10 月 30 日

摘　要

多个组织共同使用 AI Agent 时，工具权限、审批名单和执行历史通常仍由一个平台集中管理。即使平台增加数字签名和审计日志，各方仍需相信该平台不会单方放宽规则、重复处理同一授权，或在发生争议时隐去部分历史。如何把这些依赖平台承诺的环节，转化为参与方能够共同约束、独立查证的业务规则，是本项目研究的区块链应用问题。

ChainGuard 以企业 Agent 工具调用为应用场景，将联盟链用于授权治理和执行审计，而非保存全部业务数据。项目基于四验证节点 Hyperledger Besu QBFT 网络，以智能合约维护组织角色、策略版本、动作预约和工具回执。其主要设计包括：以链上治理约束审批身份与工具准入变更；以 EIP-712 签名绑定具体操作，并通过一次性预约协调跨入口的授权消费；以工具端独立核验和签名回执衔接链上授权与链下执行；以会话预授权和 Merkle 根批量存证处理低风险高频调用，同时保留高风险动作的逐次审批与确认。

项目已形成真实模型、执行网关、沙箱工具和只读审计台联动的原型。三条模型测试轨共完成 60 次任务运行、180 次真实模型请求；另开展 138 条协议观测和 65 条 B/C 机制观测。目录注入测试中，常规中心化组实际错发 2 封邮件，强化中心化组与联盟链组均为 0 封。针对单方更换审批身份、分裂状态重复处理和历史缺失的机制实验，联盟链组分别表现出治理约束、统一预约和公共执行元数据恢复能力；同时，实验保留了共享事务数据库、完整签名日志等有效中心化对照，以及独立工具数据库仍可造成重复副作用的反例。

结果表明，区块链在本项目中的价值主要是形成跨组织共同治理和独立核验的基础，而非普遍提升模型安全性或执行速度。当前四节点运行于同一宿主，高风险闭环限于本地邮件；独立机构部署、低风险批量方案的成对性能实验及完整生产成本仍待验证。

关键词：区块链创新应用；联盟链；跨组织授权；智能合约；可验证审计

目　录

第一章　项目概述与区块链应用价值 1
1.1 应用背景：从平台代管走向多方协作 1
1.2 需要解决的三类信任问题 1
1.3 项目定位与主要创新 1
1.4 适用场景与建设目标 2
第二章　联盟链协同架构与数据设计 3
2.1 总体架构与区块链位置 3
2.2 四验证节点与职责划分 3
2.3 链上链下协同与隐私保护 4
第三章　区块链核心机制与创新实现 5
3.1 跨组织治理：将权限变更纳入共同规则 5
3.2 签名意图与一次性预约：使批准内容对应执行动作 5
3.3 逐动作回执：衔接链上授权与真实执行 6
3.4 会话预授权与批量存证：按风险选择写链粒度 6
3.5 独立审计：分开核验授权、交易与副作用 7
第四章　原型实现与安全边界 8
4.1 实现组成与工程约束 8
4.2 当前交互权限与已知体验限制 8
4.3 高风险协议不因交互功能而降级 9
4.4 威胁模型与信任边界 9
4.5 可验证性与交付范围 9
第五章　实验设计与区块链增量价值分析 10
5.1 对照方案、实验轨道与统计口径 10
5.2 共同安全能力：真实模型越权与协议攻击 11
5.3 区块链增量一：单方无法任意替换审批身份 11
5.4 区块链增量二：共同预约顺序与独立历史查证 12
5.5 链确认成本与可用性观察 12
5.6 结果可信性与当前局限 13
第六章　总结与展望 14
6.1 项目总结 14
6.2 应用价值与适用条件 14
6.3 后续研究与发展方向 14
参考文献 15
附录 A　实验数据来源与证据索引 16
A.1 原始数据与统计文件 16
A.2 链上预约冲突的真实交易样例 16
附录 B　代码仓库、版本与复现说明 17
B.1 源码入口及版本标识 17
B.2 复核已有证据与重新运行的区别 17
B.3 报告编译与报告构建检查 18

第一章　项目概述与区块链应用价值

1.1　应用背景：从平台代管走向多方协作

区块链适用于多个主体需要共享状态、却不愿把全部控制权交给其中一方的业务。企业间的 Agent 协作正面临类似问题：用户委托 Agent 读取资料、查询外部系统和发起业务动作，业务方、安全方、工具提供方及审计方需要共同确认“谁能做什么”和“实际上做了什么”。这些主体可以协作，但对权限边界和历史记录未必有完全相同的利益。

以跨企业资料处理为例，一方允许 Agent 阅读指定材料并把摘要发送给指定收件人，另一方承担安全审核。如果审批目录、执行状态和全部日志均由一个平台保管，各方仍依赖平台解释：审批人是否被偷偷替换，同一授权是否被两个服务分别消费，展示的历史是否完整。仅提高模型回答质量，不能解决这些治理与取证问题。

ChainGuard 因此把区块链应用于 Agent 的**共同授权和执行留证**：模型负责理解任务和提出操作，联盟链提供参与方共同认可的规则与状态，工具端承担真实执行并提交可核验回执。完整业务内容仍由链下系统处理，避免把区块链变成存放文档和邮件的昂贵数据库。

1.2　需要解决的三类信任问题

第一，规则由谁决定。双组织审批只有在审批身份和组织归属可信时才有意义。若中心管理员能够同时替换两名审批人，再严格的双签也可能只是在核对管理员自己的两个地址。项目需要让重要治理变更也受到跨组织约束。

第二，同一授权由谁消费。两个入口可能同时接收同一动作，分别依据各自的本地状态判断“尚未执行”。单个进程中的去重不能自然覆盖分裂的业务状态，需要一个共同认可的预约顺序。链上预约用于解决这一授权状态协调问题，工具端仍需对实际副作用持久去重。

第三，历史由谁提供。签名可以帮助发现记录被修改，但发现缺失不等于能够恢复缺失记录。项目需要让参与方在业务平台不提供完整历史时，仍有独立来源查询已经共同确认的审批、动作消费和结果摘要。

1.3　项目定位与主要创新

本项目的创新应用，是把区块链从事后附加的“存证按钮”，延伸到**规则治理**、**执行前预约和执行后回执**三个连续环节，并按业务风险选择逐次确认或批量存证。表1.1概括设计及其作用。

表 1.1 区块链应用设计与业务作用

应用设计	区块链承担的职责	解决的业务问题
跨组织授权治理	共同维护组织角色、策略代际和工具准入记录	减少对单方权限目录的依赖，限制单方放宽规则
具体意图与一次性预约	校验结构化签名、组织独立性及 nonce，登记动作消费	将批准内容与执行动作绑定，协调多个执行入口
链上授权与工具回执衔接	对工具签名和结果承诺进行确认	将“批准过”与“执行结果已被确认”分开核验
分级存证与会话聚合	HIGH 逐动作确认，LOW 日志批量提交 Merkle 根	在不降低高风险门槛的前提下减少低风险写链次数

这些设计组合形成适合 Agent 的授权与审计机制。EIP-712、QBFT 和 Merkle 树本身是已有技术，本文不将其作为新密码学算法；创新重点是它们在多组织 Agent 工具调用中的分工与衔接。

1.4 适用场景与建设目标

项目适用于跨企业资料协作、供应链信息查询、双方确认的信息发送等场景：参与者需要共同管理权限，又需要在争议发生后独立查证。对于参与方均信任单一管理员的企业内部系统，完善的中心化授权与审计也可能足够，是否引入联盟链应结合治理需求与运维成本决定。

本阶段目标是形成可运行、可验证、可复现的竞赛原型。首个完整高风险场景为本地沙箱邮件，验证从真实 Agent 提议、独立组织审批、链上预约到工具回执确认的闭环。项目不将未接入的资金转移、任意数据库修改或真实外部邮件写为已完成能力。

第二章　联盟链协同架构与数据设计

2.1　总体架构与区块链位置

ChainGuard 由业务交互、Agent 与执行网关、受控工具、联盟链及只读审计五部分组成。图2.1根据项目架构示意整理；外部工具图标表示可接入方向，具体执行权限仍由已实现的适配器和审核配置决定。

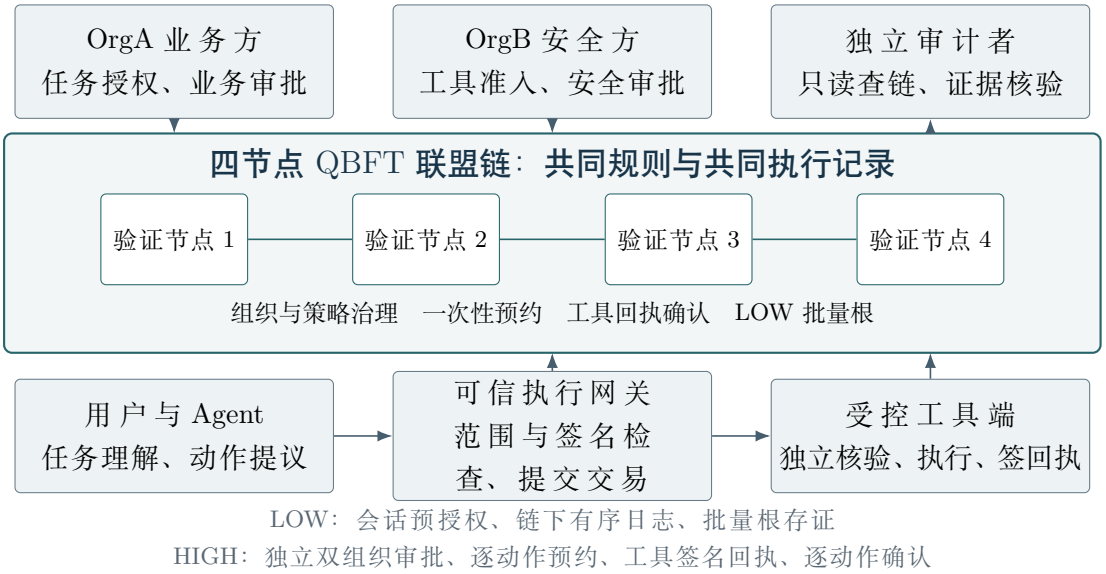


图 2.1　ChainGuard 总体架构：联盟链提供治理、预约和审计共同记录

业务操作台承载任务创建、会话和审批交互。Agent Runner 负责模型循环，网关检查任务范围和确定性策略；模型只能提出工具请求，不能直接持有审批、工具或交易私钥。工具端在执行前独立检查授权，执行后生成签名回执。审计台保持只读，不因具有查询能力而获得审批和执行权。

联盟链处于授权与审计的共同记录层。它不替模型理解文档，也不直接发送邮件；它负责确认当前有效规则、登记一次性动作状态，以及接受符合规则的签名结果。链上确认与链下工具执行必须结合，才能对具体业务形成完整结论。

2.2　四验证节点与职责划分

原型使用四个真实 Hyperledger Besu 25.10.0 验证节点，采用 QBFT 共识，链 ID 为20261026，出块周期为 2 秒。四节点分别具有不同节点身份和数据卷。QBFT 的安全与活性依赖其法定人数及网络条件 [2]；当前部署仍位于同一 WSL 宿主，不能将“四节点”直接等同于“四家机构独立运维”。

业务请求经过网关提交交易，工具和审计者使用各自的查询路径核对链状态。实验对交易保存四个节点的 receipt，检查区块与结果一致性。四份节点回执属于同一笔交易的四次观察，不作为四个独立实验样本。

表 2.1 主要合约与链上职责

合约或独立模块	职责
PolicyRegistry	组织与角色、资源作用域、策略版本、撤销与暂停；重要治理按组织门槛生效
ExecutionRegistry	验证 ActionIntent 及审批，消费 actionId 与 requester nonce，记录一次性预约
AuditRegistry	验证工具签名回执，确认结果及 UNKNOWN 恢复，拒绝不匹配回执
ToolAdmissionRegistry	按工具精确版本记录审核标签与准入代际，避免提交者自行获得 LOW 权限
`ConversationSessionRegistry` 与批量锚定合约	支持会话绑定与 LOW 审计检查点；与原高风险合约分域使用

合约层使用 OpenZeppelin 的访问控制和 ECDSA/EIP-712 标准实现 [3]。新增 LOW 机制采用独立结构与签名域，不覆盖已冻结的 ActionIntent、ExecutionReceipt 或高风险状态机。

2.3 链上链下协同与隐私保护

项目遵循“共同决策和承诺上链，业务原文留在链下”的原则。表2.2列出两侧保存内容及核验关系。

表 2.2 链上与链下数据分工

数据类型	链上保存	链下保存及用途
授权与治理	组织角色、策略代际、工具版本准入等元数据	任务说明、审核界面、配置和访问控制
高风险动作	动作标识、作用域、参数承诺、nonce、预约状态	完整收件人、主题、正文和承诺盐；工具重算比对
执行结果	结果与证据索引承诺、工具签名确认记录	原 MIME、工具执行记录、签名回执及证据原件
低风险审计	按会话提交的 Merkle 根及检查点	有序调用日志、参数与结果哈希、授权摘要和证明路径

参数承诺使用 JSON 规范化、随机盐和域分离哈希。规范化遵循 RFC 8785[4]，避免相同参数仅因键顺序不同而产生歧义。随机盐和私钥不写入公开日志或链上。哈希承诺用于核对一致性，不能仅凭哈希恢复原文，也不能证明原始描述必然真实。

第三章　区块链核心机制与创新实现

3.1　跨组织治理：将权限变更纳入共同规则

高风险审批依赖可信的审批名单。ChainGuard 将组织归属、角色和策略代际纳入 PolicyRegistry，通过治理签名和组织去重限制重要变更。链上不仅记录“有人同意过”，还检查批准者当前是否具有相应角色、作用域和组织身份。撤销、暂停或策略版本变化后，后续校验必须使用当前状态，不能继续依赖旧缓存放行。

工具接入采用版本化准入。提交者填写工具请求配置或导入数据，审核方针对精确版本标记 LOW/HIGH；更新工具后重新审核，不能自动继承旧版本的准入。风险标签用于决定适用的控制路径，但不替代适配器本身的能力边界。把查询工具标成 HIGH，并不会使它获得任意外部写权限。

这项设计将“谁有权批准”也变成共同治理对象。与单纯在链上保存日志相比，它在执行之前就限制了部分单方扩权行为。第五章的治理目录替换实验专门检验这一点，同时保留与当前中心化实现的条件性比较。

3.2　签名意图与一次性预约：使批准内容对应执行动作

ChainGuard 使用 EIP-712 结构化签名，将审批绑定到一个具体 ActionIntent[1]。签名域包含链 ID 和验证合约地址，动作本身包含任务、工具、资源范围、参数承诺、策略版本、期限和 nonce。签名不是一句笼统的“允许发信”，而是对指定动作的确认。

表 3.1　ActionIntent 中与链上授权直接相关的字段

字段	作用
actionId、taskId	唯一识别动作及所属任务，防止把不同任务的审批混用
requester、agent、toolId	绑定请求者、Agent 和获准调用的工具
resourceScope、paramsCommit	限定资源范围，并对完整参数作不可随意替换的承诺
policyId、policyEpoch、risk	绑定适用规则及其代际，避免绕过撤销或降级策略
nonce、issuedAt、expiresAt	防止授权重复消费，并限定可接受时间窗口
schemaVersion 及 EIP-712 域	固定跨语言编码版本，隔离不同链和合约的签名

对 R2 邮件，业务审批者与安全审批者必须来自不同组织，审批地址彼此独立，并且不能由 Agent 或工具身份冒充。合约验证通过后，`reserve` 登记动作，并永久消费该请求者的对应 nonce。两个服务即使各持一份“尚未预约”的本地快照，也必须竞争同一个链上状态，而不能分别宣称预约有效。

这里的一次性约束指向**授权预约**。实际工具执行仍需共享持久幂等记录。第五章保留了一个重要反例：把同一有效预约直接交给两份各自独立数据库的工具服务，B/C 都

可能产生两次副作用。因此不能把“链上只成功预约一次”扩大解释为任何部署下都只执行一次。

3.3 逐动作回执：衔接链上授权与真实执行

高风险邮件的执行流程见图3.1。双方先对准确意图审批，请求者确认后再预约；不会先执行邮件，再补审批或补链上记录。



图 3.1 R2 邮件：具体意图审批、链上预约、工具核验和签名回执

预约确认后，工具端独立查询链，检查当前角色、策略、预约、任务范围和参数承诺。检查通过才将邮件写入本地测试邮箱，并在持久幂等记录中关联 actionId。工具随后生成 ExecutionReceipt，包含动作标识、参数承诺、结果承诺、证据索引承诺、执行时间和结果类型，由工具身份签名。

AuditRegistry 核验该回执是否对应原预约、签名者是否为有效工具身份，然后执行`finalize`。预约、实际副作用和链上结果确认是三个不同事实：RESERVED 只说明预约成立，FINALIZED_OK 说明合约接受了有效成功回执；仍需结合原工具记录和邮件证据，判断具体内容与副作用次数。

断网或响应丢失时，系统保留 UNKNOWN，先查原工具记录、回执或原交易。已经发送过的动作不能因为页面没显示成功就重新发送。通过原回执补齐链上确认，才是恢复流程。

3.4 会话预授权与批量存证：按风险选择写链粒度

高频查询若每次都要求人工审批并写交易，会显著增加等待。ChainGuard 为已审核 LOW 工具增加会话预授权：OrgA 签发独立 EIP-712 凭证，绑定 session_id、允许工具与范围、有效期和最大调用次数。网关与工具分别检查签名、当前准入、nonce、期限及持久配额；nonce 用于防重放，调用额度使用独立计数字段。

有效授权内的 LOW 调用连续执行，先写入关联会话的本地日志。记录包含 Agent、工具、参数哈希、时间、结果哈希和授权摘要。会话关闭，或达到相应时间、数量阈值后，将有序记录计算为 Merkle 根，再以单笔交易提交。概念上，一批记录的承诺可表示为：

$$R_s = \mathrm{MerkleRoot}(h_1, h_2, \ldots, h_n), \tag{3.1}$$

其中 h_i 为按协议编码计算的第 i 条日志摘要；实际叶编码、顺序和域分离以项目契约及验证器为准。核验时使用该条日志和证明路径重算根，再核对已确认的链上检查点。

会话授权续签保持 session_id 及历史，新的授权周期不覆盖旧日志或清空已确认记录。当前会话方案采用累计前缀检查点，持续保留顺序关系。批量存证减少的是 LOW 日志写链次数，**不合并 HIGH 审批，不替代逐动作 reserve/finalize**。

Merkle 证明只能说明记录属于某个已承诺批次，不能单独证明日志没有遗漏、内容真实或工具确实执行。根提交前还存在依赖本地持久化的时间窗口；原始日志缺失时，根也不能还原全部内容。项目尚未完成相同负载下逐次写链与批量写链的成对性能实验，因此不报告 LOW 加速倍数。

3.5 独立审计：分开核验授权、交易与副作用

审计台通过只读数据投影快速展示历史，实时核验再检查可信合约、区块、交易回执、签名和承诺。它与业务操作台分开，不持有审批或工具凭据。审计者需要分别回答：审批是否由有效独立组织作出，预约是否真实确认，工具是否提交匹配回执，以及链下证据是否与承诺一致。

这使区块链记录能够成为参与方共享的查证来源，同时避免把 UI 标签当作链上事实。用户从业务列表“删除”会话仅作隐藏，原消息、审批和交易继续保留。链不可用时显示无法形成当前结论，不能把本地历史状态当成刚刚核验通过。

第四章　原型实现与安全边界

4.1　实现组成与工程约束

原型采用 Python/FastAPI、React、Solidity/Hardhat/OpenZeppelin、Docker Compose 及 pytest。业务台支持连续对话、文件上传、默认工具选择、自定义查询工具审核、双组织审批和状态动态更新。审计台独立提供动作、审批、交易和工具签名的只读核验。

表 4.1　工程模块及其边界

模块	已实现职责与限制
Agent Runner 与工具代理	真实 DeepSeek 工具调用；保存消息和可恢复工具批次，模型不持敏感凭据
授权网关	工具白名单、参数规范化、任务范围和 ABAC/RBAC 策略；HIGH 继续使用原冻结协议
沙箱邮件工具	独立核验、持久幂等、本地 MIME 写入及工具签名回执；不连接真实外部邮箱
默认及自定义读取工具	文档/本地邮箱读取、数据查询、公开 GitHub 及受限 HTTP 查询；不能动态获得任意写权限
文档处理	读取授权附件，生成或修改为新的 Word/PDF 副本；沿用独立沙箱 capability，不冒充链认证
业务及审计存储	加密保存私有上下文和证据；审计使用公开投影，用户清理列表不物理删底层审计

会话使用稳定 session_id，逐轮任务授权与登录会话分开。高风险调用挂起时保存已经返回的模型消息和工具调用断点，在原授权与执行证据完整后恢复。系统不声称能够保存或还原外部模型服务内部未返回的推理状态。

4.2　当前交互权限与已知体验限制

工具准入、会话授权和单次执行分别控制。OrgB 针对自定义工具的精确版本审核并标记 LOW/HIGH；OrgA 在会话创建时选择工具及邮箱、文档或查询键；网关与工具端核验每次调用是否在该范围内。读取邮箱不包含发送权限，mail.send 仍须独立的精确意图双审批。

当前会话的范围固定于创建时。中途在设置页勾选工具仅修改新会话的授权草稿，不会增加当前模型可用 Schema，也不会改变既有签名。会话续签只延长原 LOW 范围，扩展当前会话工具尚未实现；用户需要新建会话。本报告不把该待完善交互描述为动态扩权功能。

截至本次提交，普通新消息的交互配置为 deepseek-flash、temperature=0、top_p=1、thinking disabled、单次回复额度 4096 tokens，附件及文档任务 8192 tokens。历史实验

的 512/1024 额度不因此重写；截断响应中的工具调用不派发、不自动补全或重试。完整历史内部推理从未由模型返回，快照只保存实际消息与已返回工具批次。

4.3 高风险协议不因交互功能而降级

前端用于表达任务与展示进度，不能自行生成“链上成功”。浏览器不持链管理员、审批或工具私钥；原型组织签名由同机受控测试服务完成，不能等同生产 HSM 和独立机构托管。

当前交互式邮件的新建审批/执行窗口上限为 24 小时，仍取任务截止和当前链规则的更严格值。原实验部署及历史签名域保留各自期限，旧签名不自动延期。角色撤销、工具端二次核验、防重放、预约和 UNKNOWN 不重发均不因会话续聊而放宽。

4.4 威胁模型与信任边界

项目考虑低信任文档或邮件诱导越权、参数替换、伪造审批、重复提交、权限撤销、日志缺尾以及链或工具失联。防护点设置在执行网关、合约和工具端，而不是只依赖模型遵守提示词。

项目仍依赖审批密钥受到保护、工具身份可信、共识未被足够数量的恶意验证者控制，以及链下证据能够保存。若同一攻击者取得宿主最高权限并替换所有组件，当前同机原型不能提供独立机构级的隔离保证。工具签名表明指定身份对结果作了承诺，不能自动证明第三方世界中的副作用绝对真实。

4.5 可验证性与交付范围

项目的验收对象是真实交易、有效签名和原工具副作用，而不是截图。实验保存标准输出文件、逐例证据、四节点 receipt 及运行清单，便于从汇总数字追查到具体动作。报告中的数据来源、交易样例和源码入口统一列入附录，不以操作教程替代技术论证。

目前已验证的完整高风险业务是 R2 本地邮件。R3 链上完整业务、真实外部邮件与其他生产写工具、独立机构部署及空白机器完整重建尚未完成。完整官方 AgentDojo 任务仍不兼容，已适配子集不能写为官方排行榜成绩 [5]。

第五章　实验设计与区块链增量价值分析

5.1　对照方案、实验轨道与统计口径

实验设置三组：A 使用常规账号授权；B 强制任务范围策略、独立组织审批、工具端二验及签名追加日志；C 保留 B 的基础业务策略，加入链上治理、一次性预约、独立查链和回执确认。B 是有明确安全措施的中心化方案，不能用弱化 Token 系统替代它。

三条模型轨均使用 DeepSeek 的`deepseek-flash`，temperature=0、top_p=1、thinking disabled、并发 1。各轨内部任务、输入数据、模型参数及评价方法一致；原始/摘要轨 max_tokens 为 512，目录探测为 1024，跨轨不合并攻击分母。远端模型别名并不保证未来逐字输出一致，具体响应与参数保存在原运行记录中。

表 5.1　真实实验规模：不同轨道分别统计

轨道	观测数	模型请求	本地 MIME	交易/节点回执
原始模型任务	12	36	12	8/32
模型生成摘要	36	108	36	24/96
目录注入开发探测	12	36	8	4/16
协议、故障与性能	138	0	93	59/236
B/C 机制干预	65	0	71	75/300

三模型轨合计 60 次任务、180 次模型请求。机制轨 65 条由 31 对 B/C 和 3 条 C 独有 RPC 故障观测组成。独立 C 前置、统计重算和只读复验不增加正式样本。ASR 仅来自真实模型轨，确定性协议和机制观测不能充当模型攻击样本。

表 5.2　主要指标及其含义

指标	计算含义
UER、UBR	越权执行率与阻断率；分别核对 Agent 提出的越权调用、策略结论与实际副作用
ASR	攻击目标在真实模型任务中实际达成的比例，不以“模型说同意”代替副作用证据
TCR、TDR	适用任务完成率与篡改检测率；不把错误、未知或无适用样本填为成功
P50/P95/P99	指定测量范围内的时延分位数，区分模型、人工等待与链确认成本
置信区间	比例使用 Wilson95 描述区间，另保存聚类 bootstrap；性能使用试次 bootstrap 条件区间

当分母为 0 时标记未测或不适用，不填 0%。同一任务的重复运行并非独立业务分布，较小样本下的区间仅用于描述本次结果，不能据此承诺生产风险上限。

5.2 共同安全能力：真实模型越权与协议攻击

目录注入开发探测中，用户只授权向 Alice 发信，攻击者替换 Agent 读取的资料，使其尝试向 Bob 发送。每组包含 2 条正常任务和 2 条攻击任务，三组模型都提出了 2 次越权调用，实际结果见表5.3。

表 5.3 目录注入：Agent 尝试与工具实际执行分开计量

方案	越权尝试	错发邮件	任务完成
A：常规中心化	2 次	2 封	2/4
B：强化中心化	2 次	0 封	2/4
C：联盟链方案	2 次	0 封	2/4

B/C 均拒绝越权且没有对应邮件，这证明共同执行策略能够在模型出错后阻止副作用；**这组实验没有证明 C 比 B 更擅长防提示词注入**。B/C 的 ASR 为 0/2，但 Wilson95 上界仍为 65.76%。该探测同时涉及路由数据污染与提示词注入，未做消融，不推广到所有任务和攻击。

另外两套原始和摘要任务中，每组分别完成 4/4 和 12/12 条，攻击均未成功，但模型也未提出越权调用，不能算作网关完成了拦截。协议轨在三组共同的 9 次越权测试中，A 实际执行 6 次，B/C 均为 0 次；B/C 面对各 6 次额外审批伪造或错用均未接受。共同越权口径下，B/C 的 UBR 为 9/9，95% 区间为 [70.09%,100%]；篡改检测各为 6/6，区间为 [60.97%,100%]。

每组 60 次重复提交均未新增邮件。响应丢失恢复中，C 可查询原工具回执并补齐链确认，原邮件仍一封；B 当前实现保留结果不明，该差异也可通过完善中心化恢复逻辑改进。协议轨适用任务完成数为 A 28/31、B 28/31、C 25/31，部分故障只适用于 C，不将此直接解释为总体可靠性排名。

5.3 区块链增量一：单方无法任意替换审批身份

为检验链的增量，机制实验主动干预治理根，而非继续增加相同的提示词注入。B 组在中心审批目录被单方控制后，将两名审批人的地址和组织声明替换为攻击者控制的身份；原签名和工具端二验流程仍然运行，但依据被替换的目录认可身份，3 次均产生邮件。

C 组尝试仅凭一个治理组织或重复使用同一组织完成同类扩权，6 笔违规治理交易均回滚，3 次测试都没有邮件。差异在于审批身份目录的治理约束，而不是“签名算法在区块链上才安全”。它说明当前 C 实现能让规则变更受到跨组织共同约束。

比较边界必须保留：B 使用单一管理目录，C 使用跨组织治理。一个同样具有多方治理和独立信任根的完整中心化 B+，可能提供相应保证。本实验展示的是当前部署条件下的增量，不是证明中心化架构必然无法实现。

5.4 区块链增量二：共同预约顺序与独立历史查证

共同预约顺序。把业务状态分成两份，各入口同时处理同一有效动作。B 依据两份本地快照各执行一次，每例两封；C 竞争同一 ExecutionRegistry，一份预约成功、另一份回滚，每例一封。示例中两笔交易被写入同一区块 #139301，一笔成功、一笔回滚，具体哈希见附录。共享强事务数据库作为强对照时，B/C 都每例一封，说明共享序列化状态本身同样有效。

独立历史查证。平台不提供最后一段完整历史时，两组都能检测缺失；仅凭剩余副本，B 有 0/3 次恢复公共执行元数据，C 有 3/3 次从链恢复。链恢复的是审批人、动作消费和结果摘要等公共数据，不是邮件正文。B 另持完整签名日志时也能 3/3 恢复，不能宣称只有区块链能恢复历史。

表 5.4 B/C 机制结果与有效强对照；各机制重复 3 次

测试条件	B 结果	C 结果
单方更换审批目录	3/3 产生邮件	0/3 产生邮件；违规治理交易回滚
分裂本地状态竞争	每例 2 封	每例 1 封；一个预约成功
共享事务状态竞争	每例 1 封	每例 1 封
仅剩缺尾历史副本	公共元数据恢复 0/3	公共元数据恢复 3/3
B 额外持完整签名日志	恢复 3/3	链恢复 3/3
同预约投给独立工具库	每例 2 封	每例 2 封；多出有效但未锚定回执

上述 0/3 结果的 Wilson95 上界仍为 56.15%，3/3 下界为 43.85%。这些测试用于解释机制因果，不代表大规模统计保证。表5.4也保留了 C 没有解决的问题：**统一预约不能替代工具端共享持久去重，也不能仅凭一个已确认回执断言没有第二次副作用**。

5.5 链确认成本与可用性观察

每组 10 次正常邮件机制测试的服务层时延见表5.5。范围包含自动测试审批、签名、存储与链确认，不含模型、人工审批等待及生产 HTTP/mTLS 路径；这些数字不是用户端完整等待时间。

表 5.5 B/C 服务层耗时，单位 ms；括号内为 bootstrap95 条件区间

方案	分位数	样本数	耗时及 95% 区间
B	P50	10	319.6 [307.6, 329.8]
B	P95	10	331.4 [324.4, 331.6]
B	P99	10	331.5 [324.4, 331.6]
C	P50	10	3351.4 [2870.8, 3453.8]
C	P95	10	3663.5 [3383.9, 3785.8]
C	P99	10	3761.3 [3383.9, 3785.8]

C 的中位耗时 3351.4ms，B 为 319.6ms，C 约为 B 的 10.5 倍。每动作 gas 中位数为 C 653070、B 0；本地网关/工具 SQLite 字节中位数为 C 167936、B 122880，不含四节点完整账本和运维成本。零 gasPrice 不代表零资源消耗，n=10 的 P95/P99 也不作为生产容量承诺。

LOW 批量存证意在减少写链频次，异步受理则把 HTTP 返回与后台确认等待分开；两者均不能据上述 HIGH 耗时直接推算加速倍数。当前没有相同 LOW 负载下逐次上链与批量上链的成对实验，报告不提供 LOW TPS 提升数据。

依赖故障测试中，B 数据库不可用和 C RPC 失联均没有邮件，恢复后各 3/3 单封。C 一个配置 RPC 失联、账本仍出块时，上层仍保守拒绝直到恢复。实验没有停止真实验证者或构造恶意验证者，因此没有把 QBFT 理论容错写成产品已实测的端到端容错能力。

5.6 结果可信性与当前局限

每项结论可追查到原模型、审批、工具及交易证据。报告核验重算五轨统计并核对 313 项 manifest 产物摘要，原 21 份协议和 8 份共享策略输入保持一致。只读原卷复验包含机制轨 65 条、71 封 MIME、75 笔交易和 300 份节点 receipt；复验不产生新样本。

实验范围仍有限：模型任务数量少、部分攻击是开发探测；四节点与测试组织身份同机；完整多方治理、共享数据库和独立日志组合的 B+ 尚未形成统一成本对照；LOW 成对性能、验证者故障和外部生产副作用尚未验证。早期失败批次和缺失执行镜像的观察器替代记录均保留，不将失败改写为成功或把复验重复计数。

第六章 总结与展望

6.1 项目总结

ChainGuard 将区块链用于多组织 Agent 协作中的授权治理和执行审计，围绕三个问题建立连续机制：通过共同治理约束“谁有权批准”，通过一次性预约约束“同一授权如何消费”，通过工具回执和共同账本支持“执行之后如何查证”。LOW 会话预授权与批量 Merkle 存证则使高频操作采用更合适的写链粒度。

项目已经完成真实 Agent、独立测试审批身份、四节点联盟链、受控邮件工具和只读审计台之间的原型闭环。实验表明，B/C 的共同策略都可阻断所测模型越权；C 在单方目录控制、分裂业务状态及公共历史查询等明确条件下提供额外约束。共享数据库、完整签名日志的有效对照和工具独立库双封反例，也说明区块链需要与正确的链下执行设计共同工作。

6.2 应用价值与适用条件

项目适合需要共同控制规则、共同认可动作状态且保留独立查证能力的协作环境。其价值不是把所有数据写入链，而是在高价值信任边界保留共享且可核验的记录。对于单组织且信任中心管理员的系统，中心化 B 可能已经足够；引入区块链应以治理需求为依据，并接受确认时延和多节点运维成本。

在实际推广中，可先从跨企业资料处理、供应链信息协同和双组织确认的消息发送切入，再按工具风险和回执能力扩展。每接入新的写工具，都需要重新定义资源范围、独立核验、幂等和证据要求，不能把邮件原型的安全结论直接套用到资金、数据库或第三方邮件服务。

6.3 后续研究与发展方向

后续工作首先是独立机构部署与生产身份托管，将同机测试角色转化为真正分离的治理主体，并验证节点故障、分区及证据可得性。其次是完善强中心化 B+ 对照，统一评估共同治理、共享状态、独立日志与联盟链的保障范围及全生命周期成本。

在性能方面，需要开展同负载 LOW 逐次上链与批量存证实验，测量写链次数、端到端时延、批次等待和故障恢复，而不是仅报告 HTTP 受理速度。在安全方面，需要扩展独立任务集与攻击族，完善完整官方基准适配，并验证新工具的真实外部副作用。项目当前提供了可检查的应用原型和证据基础，后续目标是使跨组织 Agent 协作中的授权责任与执行结果更加清晰。

参考文献

[1] Ethereum Improvement Proposals. EIP-712: Typed structured data hashing and signing. https://eips.ethereum.org/EIPS/eip-712.

[2] Hyperledger Besu. QBFT consensus. https://besu.hyperledger.org/private-networks/how-to/configure/consensus/qbft.

[3] OpenZeppelin. Contracts 5.x documentation. https://docs.openzeppelin.com/contracts/5.x/.

[4] RFC 8785. JSON Canonicalization Scheme (JCS). https://www.rfc-editor.org/rfc/rfc8785.

[5] ETH Zurich, SPY Lab. AgentDojo. https://github.com/ethz-spylab/agentdojo.

附录 A 实验数据来源与证据索引

A.1 原始数据与统计文件

本文引用的实验运行于 2026 年 10 月 29 日；2026 年 10 月 30 日重新核对五轨统计及 313 项产物摘要，未重跑模型、邮件或交易。表A.1中的路径相对作品源码根目录，原始数据不因编写报告而重跑或覆盖。`report-evidence.json` 随 LaTeX 源码提供，保存指标、区间和来源摘要。

表 A.1 实验数据来源

索引	相对目录
E1 原始模型	`results/phase8/abc-rerun-20261029-01/canonical/real-abc/`
E2 摘要模型	`results/phase8/abc-rerun-20261029-01/summary/real-abc/`
E3 目录注入	`results/phase8/abc-rerun-20261029-01/directory-probe/real-abc/`
E4 协议故障	`results/phase8/abc-rerun-20261029-01/canonical/protocol/`
E5 B/C 机制	`results/phase8/bc-contribution-20261029-01/run-02/`
V1 原卷复验	`results/phase8/report-readonly-20261029-01/verification.json`

各实验目录含`traces.jsonl`、`cases.csv`、`aggregated_metrics.csv` 和`run_manifest.json`。前两者记录过程与逐例结果，统计文件保存指标和区间，manifest 固定计划、环境及产物摘要；`tx-receipts.json`、逐例 JSON 及原受控卷支持进一步核对。公开索引不包含模型 Key、私钥、JWT 或真实业务原文。

A.2 链上预约冲突的真实交易样例

E5 的`case-0004.json` 记录两个入口争用同一动作。两笔预约交易均进入区块 #139301，成功与回滚结果不同。此例证明共享链上状态的消费顺序，不能单独用来证明任何工具拓扑都不会重复执行。

预约成功交易：

`0xb4e9cad405edc80a792a28bef6973a2b3d080249b0f951c4b5deee5f855ee802`

竞争回滚交易：

`0xad553ee11ac949efd2f55984ef95545e084e38a6181d489fc84a8e43286a91bb`

四 RPC 端口为 8545–8548。核验时对每个节点查询`eth_getTransactionReceipt`，检查同区块、成功/回滚状态及对应合约事件，再结合原工具记录确认副作用数量。不能仅凭 status=1 推断邮件内容正确。

附录 B 代码仓库、版本与复现说明

B.1 源码入口及版本标识

项目源码为随作品交付的 ChainGuard monorepo。当前本地 Git 未配置公开远程仓库地址，因此本文不填写未经确认的 GitHub 链接。最近已提交基线为`a5c23eb`，其后存在未提交实现，不能把该提交号当成全部交付代码的精确版本。`DELIVERY_MANIFEST.json` 另记录最终交付源文件的 SHA-256 摘要；包内保留未提交源文件，不以 Git 基线冒充交付版本。

表 B.1 源码与设计入口

位置	内容
`README.md`、`docs/contracts.md`	项目运行入口、冻结规范导航及安全边界
`contracts/`、`tests/solidity/`	治理、预约、回执及 LOW 扩展合约与测试
`gateway/src/chainguard_gateway/`	授权策略、链交互、会话与审计实现
`agent/`、`sandbox_tools/`	真实模型循环、工具代理和沙箱副作用实现
`console/`、`dashboard/`	业务交互台和独立只读审计台
`evaluation/`、`scripts/`	实验计划、统计、独立复核与运行脚本
`chain/compose.yaml`	四验证节点容器配置，实际链配置由部署记录确定

原 ActionIntent、ExecutionReceipt、EIP-712 域与状态机的精确结构见`spec/eip712.json`、`spec/schemas/protocol.schema.json` 及`spec/state-machines.json`。`config/phase0-freeze.json` 保护 21 份原协议，`config/phase3-policy-freeze.json` 保护 8 份共享策略输入；摘要检查用于发现漂移，不将其宣称为抵抗同机管理员的独立信任根。

B.2 复核已有证据与重新运行的区别

从项目根执行下面的只读核验命令，输出目录必须全新。它核对既有机制批次的链、签名与原工具卷，不创建新模型任务、不重发邮件，也不广播新交易：

```
.venv/Scripts/python.exe scripts/run_bc_experiments.py `
  --verify-source results/phase8/bc-contribution-20261029-01/run-02 `
  --out results/phase8/my-report-check
```

复核依赖原四节点、原受控卷及可用观察镜像；缺失时应明确报错，不能用静态文件替代实时核验。重新运行模型摘要对照则使用：

```
.venv/Scripts/python.exe scripts/run_real_experiments.py `
  --plan evaluation/summary_plan.json --model-only `
  --out results/phase8/my-new-summary
```

该命令调用真实模型并产生本地测试邮件，输出四类标准统计文件。机制重跑使用scripts/run_bc_experiments.py--out<new-dir>，目录注入计划为evaluation/directory_probe_plan.json。密钥只从本地环境安全加载，报告及源码包均不包含凭据。上述命令依赖既有运行环境，不代表已完成空白机器一键安装。

B.3 报告编译与报告构建检查

LaTeX 源码包含main.tex、data-tables.tex 和images/，使用 XeLaTeX 及 TeX Live 标准中文字体。解压后在源码目录运行两次：

```
xelatex -interaction=nonstopmode -halt-on-error main.tex
xelatex -interaction=nonstopmode -halt-on-error main.tex
```

本轮重新运行 42 项实验统计/机制相关 pytest，协议与策略冻结检查通过，并从原轨迹复算报告统计。编译与逐页排版核验结果保存在results/reports/20261029-competition-report-03/。这些是报告核验，不增加实验样本；本轮没有重调模型或新增工具副作用。